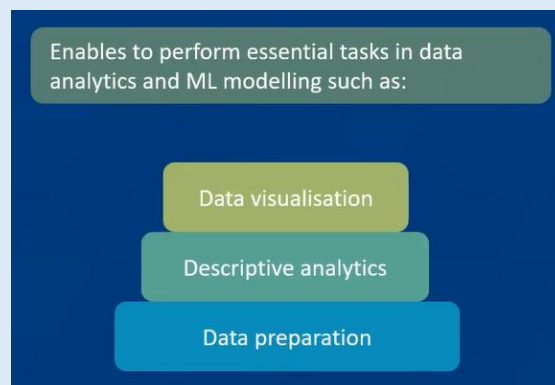


Module 6: Week 7: Introduction to Python - Part 2

Introduction to Python Libraries

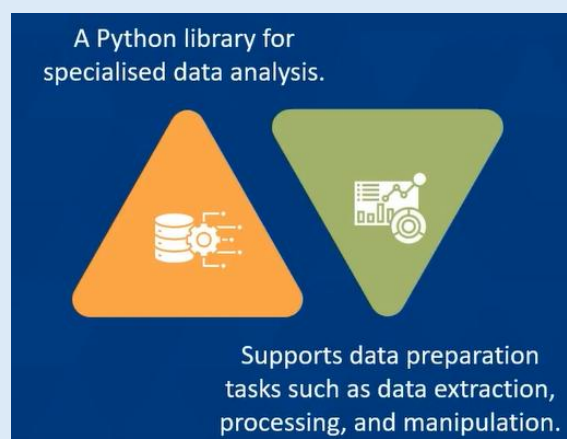
- Python offers essential high-level programming features such as control flow, functions, variables, and built-in data structures.
- Built-in Python data structures are limited in handling two-dimensional datasets required for scientific and analytical tasks.
- Domain-specific libraries address limitations by enabling efficient storage, manipulation, and analysis of complex datasets.
- These libraries support key data science functions, including data preparation, descriptive analytics, and visualisation tasks.



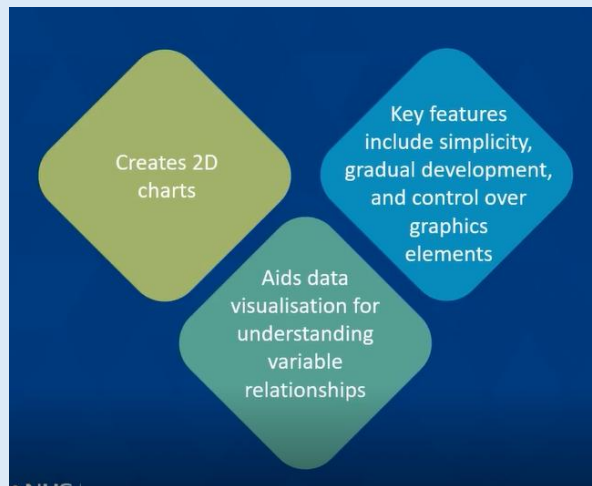
- Seamless integration among libraries creates an efficient and unified environment for data scientists and machine learning engineers.

Overview of Python Libraries for Data Science and AI

- NumPy provides the foundational ndarray data structure for scientific computing and enables efficient handling of multidimensional numerical arrays.
- Pandas offers powerful series and dataframe structures for data preparation, manipulation, and seamless integration with SQL-like capabilities in Python.



- Matplotlib supports intuitive creation and customisation of two-dimensional charts, aiding data visualisation and exploration within analytical workflows.



- Scikit-Learn and TensorFlow enable development of machine learning and deep learning models, supporting tasks from training to evaluation and deployment.
- Python's broader ecosystem includes tools like OpenCV, SpaCy, PySpark, and MLflow, covering vision, NLP, big data, and end-to-end machine learning lifecycle management.



Introduction to NumPy ndarray

- NumPy's ndarray is a homogeneous, multidimensional array where all elements share the same data type and memory size.
- Attributes such as shape, size, rank, and data type provide essential metadata about the structure and contents of the ndarray.

```
import numpy as np

a = np.array([[1.3, 2.4], [0.3, 4.1]])

print('a =\n', a)
print('type(a) =', type(a))
print('a.dtype =', a.dtype)
print('a.ndim =', a.ndim)
print('a.size =', a.size)
print('a.shape =', a.shape)
```

Terminal Output:

```
a =
[[1.3 2.4]
 [0.3 4.1]]
type(a) = <class 'numpy.ndarray'>
a.dtype = float64
a.ndim = 2
a.size = 4
a.shape = (2, 2)
```

- Arithmetic operations between ndarrays or between an ndarray and a scalar are performed element-wise, producing a new array as output.

```
import numpy as np

a = np.arange(4) # stop at < n
print(a)
print(a+4)
print(a*4)
print('='*20)

b = np.arange(4,8) # start,stop
print(b)
print(a+b)
print(a*b)
```

Terminal Output:

```
[0 1 2 3]
[4 5 6 7]
[ 0 4 8 12]
=====
[4 5 6 7]
[ 4 6 8 10]
[ 0 5 12 21]
```

- Matrix multiplication is supported through the dot() function and allows reshaping of arrays to perform complex linear algebra operations.

```
import numpy as np

A = np.arange(0,9).reshape(3,3)
print(A)

B = np.ones((3,3))
print(B)

print(np.dot(A,B))
```

Terminal Output:

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
[[ 3.  3.  3.]
 [12. 12. 12.]
 [21. 21. 21.]]
```

- Indexing, slicing, and transposing functions offer flexible methods for accessing, updating, and traversing ndarray elements across any dimension.

Data Processing with NumPy ndarray

- Boolean indexing allows flexible data selection and conditional assignment by using logical and relational operators on ndarrays.

```
import numpy as np

names = np.array(['Bob', 'Joe', 'Will', 'Bob', 'Will',
                  'Joe', 'Joe'])

data = np.random.randn(7, 4)

print(names)
print(data)
print('='*20)

print(names == 'Bob')
print('='*20)

print(data[names == 'Bob'])
```

Terminal Output:

```
['Bob' 'Joe' 'Will' 'Bob' 'Will' 'Joe' 'Joe']
[[-0.89577974  1.54269801 -0.65710122 -0.02106095]
 [-0.07219851  2.59867582 -1.17149883 -0.11310034]
 [-0.93932739 -0.60388498  0.47375245 -0.35778667]
 [ 0.22973673 -0.97535174 -0.9474152  0.33678001]
 [-1.92729262 -0.25822306  0.25997436 -1.48631904]
 [-0.90368837  1.58820552 -0.24064966  1.33621901]
 [-0.90547287 -2.08150753  1.11944146 -0.89368251]]
=====
[ True False False  True False False False]
=====
[[-0.89577974  1.54269801 -0.65710122 -0.02106095]
 [ 0.22973673 -0.97535174 -0.9474152  0.33678001]]
```

- Universal functions (ufuncs) enable fast, elementwise operations on ndarrays and are more efficient than equivalent Python code.
- Vectorised operations replace loops with array expressions, significantly improving the performance of numerical computations.



- The `np.where` function and built-in aggregation methods like `sum`, `mean`, and `std` support concise conditional logic and statistical analysis.

```
import numpy as np

xarr = np.array([1.1, 1.2, 1.3, 1.4, 1.5])
yarr = np.array([2.1, 2.2, 2.3, 2.4, 2.5])
cond = np.array([True, False, True, True, False])

result = np.where(cond, xarr, yarr)
print(result)
```

Terminal Output:

[1.1 2.2 1.3 1.4 2.5]

Method	Description
<code>sum</code>	- Sum of all the elements in the array or along an axis - Zero length arrays have sum 0
<code>mean</code>	- Arithmetic mean - Zero length arrays have NaN mean
<code>std, var</code>	Standard deviation and variance, with optional degree of freedom adjustment (default denominator n)
<code>min, max</code>	Minimum and maximum
<code>argmin, argmax</code>	Indices of minimum and maximum elements, respectively
<code>cumsum</code>	Cumulative sum of elements starting from 0
<code>cumprod</code>	Cumulative product of elements starting from 1

- The `numpy.random` module generates large arrays of random samples from various distributions more efficiently than Python's built-in random module.

Introduction to Pandas Series

- NumPy ndarrays are limited by their homogeneous data type and reliance on zero-based indexing for data access.

Homogeneous data structure: All elements must be the same type

Zero-based indexing: Difficult to manage large datasets

Need for labels: Text labels are more intuitive than index numbers

- Pandas Series offers a one-dimensional labelled data structure where each element can be accessed using meaningful text labels.
- Series supports element selection, assignment, and filtering using both zero-based indexing and custom labels through `iloc` and `index`.
- Series enables descriptive analytics by providing methods to examine unique values, value counts, and the presence of specific entries.

```
import pandas as pd

serd = pd.Series([1,0,2,1,2,3],
index=['white','white','blue','green','green','yellow'])
print(serd)
print('='*20)

# All unique values without duplicates
print(serd.unique())
print('='*20)

# Return unique values with counting of occurrences
print(serd.value_counts())
print('='*20)

# Evaluates values membership
print(serd.isin([0,3]))

print(serd[serd.isin([0,3])])
```

Terminal Output:

```
white    1
white    0
blue     2
green    1
green    2
yellow   3
dtype: int64

=====
[1 0 2 3]
=====
1 2
2 2
0 1
3 1
Name: count, dtype: int64
=====
white    False
white    True
blue     False
green    False
green    False
yellow   True
dtype: bool
white    0
yellow   3
dtype: int64
```

- Arithmetic operations between Series result in aligned data based on labels, with missing matches producing NaN to indicate absent values.

```
import pandas as pd

mydict1 = {'red':2000, 'blue': 1000, 'yellow':500, 'orange':1000}
myseries1 = pd.Series(mydict1)
print(myseries1)
print('='*20)

mydict2 = {'red':400, 'yellow':1000, 'black':700}
myseries2 = pd.Series(mydict2)
print(myseries2)
print('='*20)

myseries3 = myseries1 + myseries2
print(myseries3)
```

Terminal Output:

```
red      2000
blue     1000
yellow    500
orange   1000
dtype: int64

=====
red       400
yellow   1000
black     700
dtype: int64

=====
black     NaN
blue      NaN
orange    NaN
red     2400.0
yellow   1500.0
dtype: float64
```

Introduction to Pandas DataFrame

- A Pandas DataFrame is a two-dimensional, labelled data structure that can hold mixed-type data across multiple columns.
- Each row and column in a DataFrame is indexed, enabling structured access similar to spreadsheets or relational database tables.

DataFrame			
Index	Columns		
	Colour	Object	Price
0	Blue	Ball	1.2
1	Green	Pen	1.0
2	Yellow	Pencil	0.6
3	Red	Paper	0.9
4	White	Mug	1.7

- A DataFrame can be created from a dictionary of Series, where keys define column names and values define column data.

```
import pandas as pd

data = {'color': ['blue', 'green', 'yellow', 'red', 'white'],
        'object': ['ball', 'pen', 'pencil', 'paper', 'mug'],
        'price': [1.2, 1.0, 0.6, 0.9, 1.7]}

print(data)
print('='*20)

frame = pd.DataFrame(data)
print(frame)
```

Terminal Output:

```
{'color': ['blue', 'green', 'yellow', 'red', 'white'], 'object': ['ball',
'pen', 'pencil', 'paper', 'mug'], 'price': [1.2, 1.0, 0.6, 0.9, 1.7]}
=====
   color object price
0  blue   ball   1.2
1 green   pen   1.0
2 yellow pencil   0.6
3  red   paper   0.9
4 white    mug   1.7
```

- Element selection is flexible using column names, iloc for positional indexing, and loc for label-based indexing at row and cell level.

```
import pandas as pd

data = {'color': ['blue', 'green', 'yellow', 'red', 'white'],
        'object': ['ball', 'pen', 'pencil', 'paper', 'mug'],
        'price': [1.2, 1.0, 0.6, 0.9, 1.7]}

frame = pd.DataFrame(data, index=['a', 'b', 'c', 'd', 'e'])

print(frame['price'])
print(frame.price)
print('='*20)

print(frame.iloc[0])
print(frame.iloc[0]['price'])
print(frame.iloc[0].price)
print('='*20)

print(frame.loc['a'])
print(frame.loc['a']['price'])
print(frame.loc['a'].price)
```

Terminal Output:

```
a    1.2
b    1.0
c    0.6
d    0.9
e    1.7
Name: price, dtype: float64
a    1.2
b    1.0
c    0.6
d    0.9
e    1.7
Name: price, dtype: float64
=====
   color  object  price
a    blue   ball    1.2
Name: a, dtype: object
1.2
1.2
=====
   color  object  price
a    blue   ball    1.2
Name: a, dtype: object
1.2
1.2
```

- Built-in functions support arithmetic operations, handling of missing data as NaN, and statistical summaries using methods like describe().

Data Preparation with Pandas DataFrame

- Data import from formats like CSV, Excel, SQL, and JSON made simple using Pandas functions such as read_csv() and read_sql_query().

```
import pandas as pd

csvframe1 = pd.read_csv('../data/myCSV_01.csv')
print(csvframe1)
print('='*20)

print(csvframe1.describe())
print('='*20)

csvframe2 = pd.read_csv('../data/myCSV_01.csv', index_col=4)
print(csvframe2)
print('='*20)
```

Terminal Output:

	white	red	blue	green	animal
0	1	5	2	3	cat
1	2	7	8	5	dog
2	3	3	6	7	horse
3	2	2	8	3	duck
4	4	4	2	1	mouse

```
=====
count    white      red      blue      green
mean    2.400000    4.200000    5.200000    3.800000
std      1.140175    1.923538    3.03315    2.280351
min      1.000000    2.000000    2.000000    1.000000
25%      2.000000    3.000000    2.000000    3.000000
50%      2.000000    4.000000    6.000000    3.000000
75%      3.000000    5.000000    8.000000    5.000000
max      4.000000    7.000000    8.000000    7.000000
=====
      white red blue green
animal
cat      1  5  2  3
dog      2  7  8  5
horse    3  3  6  7
duck     2  2  8  3
mouse    4  4  2  1
```

- Index assignment with `index_col` improves clarity by setting meaningful row identifiers like the animal column.
- Missing values handled effectively using `dropna()`, `isnull()`, and replacement with defaults or `NaN` for clean analysis.

```
import pandas as pd

data1 = {'ball': [0,4,8,12],
         'pen': [1,5,9,13],
         'pencil': [2,6,10,14],
         'paper': [3,7,11,15]}

frame1 = pd.DataFrame(data1,
                      index=['red', 'blue', 'yellow', 'white'])

data2 = {'mug': [0,3,6,9],
         'pen': [1,4,7,10],
         'ball': [2,5,8,11]}

frame2 = pd.DataFrame(data2,
                      index=['blue', 'green', 'white', 'yellow'])

frame3 = frame1.add(frame2)
print(frame3)
print('='*20)

print(frame3.dropna())
print('='*20)

# Keep rows with at least 2 non-NA value
print(frame3.dropna(thresh=2))
print('='*20)

# Drop those rows with NA for ball
print(frame3.dropna(subset=['ball']))
print('='*20)

print(frame3.fillna(0))
print('='*20)

print(frame3.isnull())
print('='*20)
```

Terminal Output:

```
Empty DataFrame
Columns: [ball, mug, paper, pen, pencil]
Index: []
=====
      ball mug paper pen pencil
blue    6.0 NaN  NaN  6.0  NaN
green  NaN NaN  NaN  NaN  NaN
red     NaN NaN  NaN  NaN  NaN
white  20.0 NaN  NaN  20.0  NaN
yellow  19.0 NaN  NaN  19.0  NaN
=====
      ball mug paper pen pencil
blue    6.0 NaN  NaN  6.0  NaN
white  20.0 NaN  NaN  20.0  NaN
yellow  19.0 NaN  NaN  19.0  NaN
=====
      ball mug paper pen pencil
blue    6.0  0.0  0.0  6.0  0.0
green    0.0  0.0  0.0  0.0  0.0
red       0.0  0.0  0.0  0.0  0.0
white    20.0  0.0  0.0  20.0  0.0
yellow   19.0  0.0  0.0  19.0  0.0
=====
      ball mug paper pen pencil
blue  False  True  True  False  True
green  True  True  True  True  True
red    True  True  True  True  True
white  False  True  True  False  True
yellow False  True  True  False  True
```

- New columns created and modified using vectorisation, allowing efficient feature engineering without loops or control statements.
- Data export to formats like CSV supported through `to_csv()`, enabling smooth transition to visualisation or machine learning workflows.

```
import pandas as pd
import numpy as np

csvframe = pd.read_csv('../data/myCSV_01.csv')
csvframe['total'] = csvframe['white'] + csvframe['red'] + csvframe['blue'] + csvframe['green']
csvframe['excess'] = np.where(csvframe['total'] > 15, True, False)

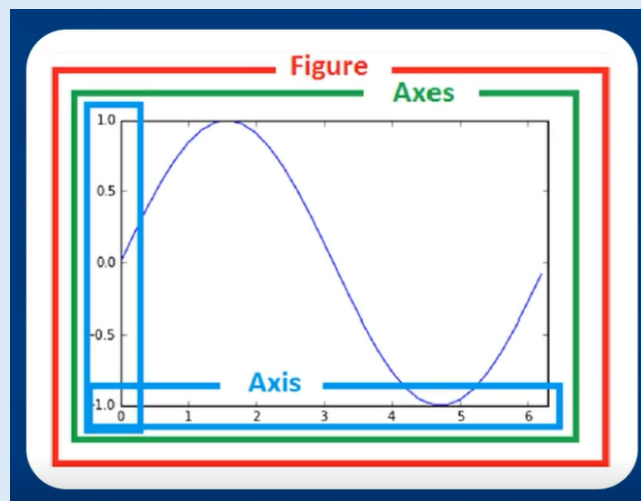
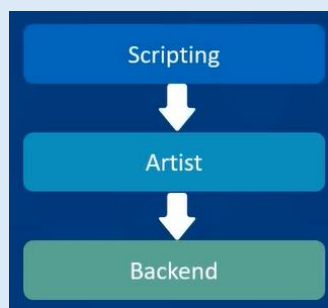
csvframe.to_csv('../data/myCSV_01_out.csv', index=True, index_label='id')
```

	A	B	C	D	E	F	G	H
1	id	white	red	blue	green	animal	total	excess
2		0	1	5	2	3 cat	11	FALSE
3		1	2	7	8	5 dog	22	TRUE
4		2	3	3	6	7 horse	19	TRUE
5		3	2	2	8	3 duck	15	FALSE
6		4	4	4	2	1 mouse	11	FALSE

Activate Windows

Introduction to Matplotlib

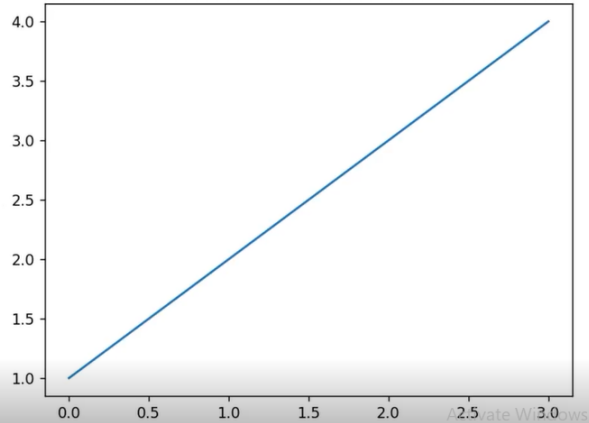
- Matplotlib architecture consists of three layers—scripting, artist, and backend—each handling chart creation and rendering in a structured hierarchy.



- Pyplot module offers command-style functions to build and customise charts, including titles, labels, legends, and axis scaling.

The pyplot module provides command-style functions to modify the figure.

```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4])
plt.show()
```



- Integration with Pandas allows direct plotting from DataFrames using plot() function, simplifying visualisation of tabular data.
- Chart customisation includes changing plot types, adding annotations, and modifying axis labels for clearer data representation.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {'Name': ['Bob', 'Jessica', 'Mary', 'John', 'Mel'],
        'Grade': [76, 83, 77, 78, 95]}

df = pd.DataFrame(data)

df.plot()

displayText = "Line Plot"
xloc = 1
yloc = df['Grade'].max()
xtext = 8
ytext = -150
plt.annotate(displayText, xy=(xloc, yloc), arrowprops=dict(facecolor='black', shrink=0.05),
             xytext=(xtext, ytext), xycoords=('axes fraction', 'data'), textcoords='offset points')

plt.show()
```

- DataFrame index manipulation enhances chart readability, especially when switching from numeric indices to meaningful labels like names.

```
import pandas as pd
import matplotlib.pyplot as plt

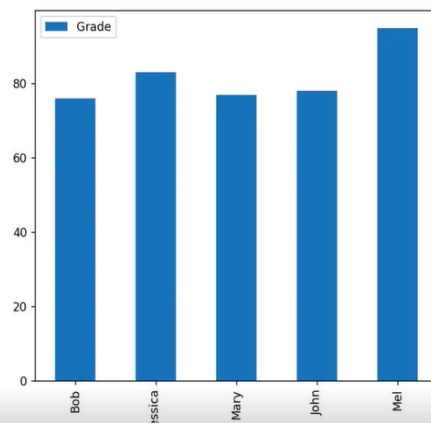
data = {'Name': ['Bob', 'Jessica', 'Mary', 'John', 'Mel'],
        'Grade': [76, 83, 77, 78, 95]}

df = pd.DataFrame(data)
df = df.set_index(df['Name'])
print(df)

df.plot(kind='bar')
plt.show()
```

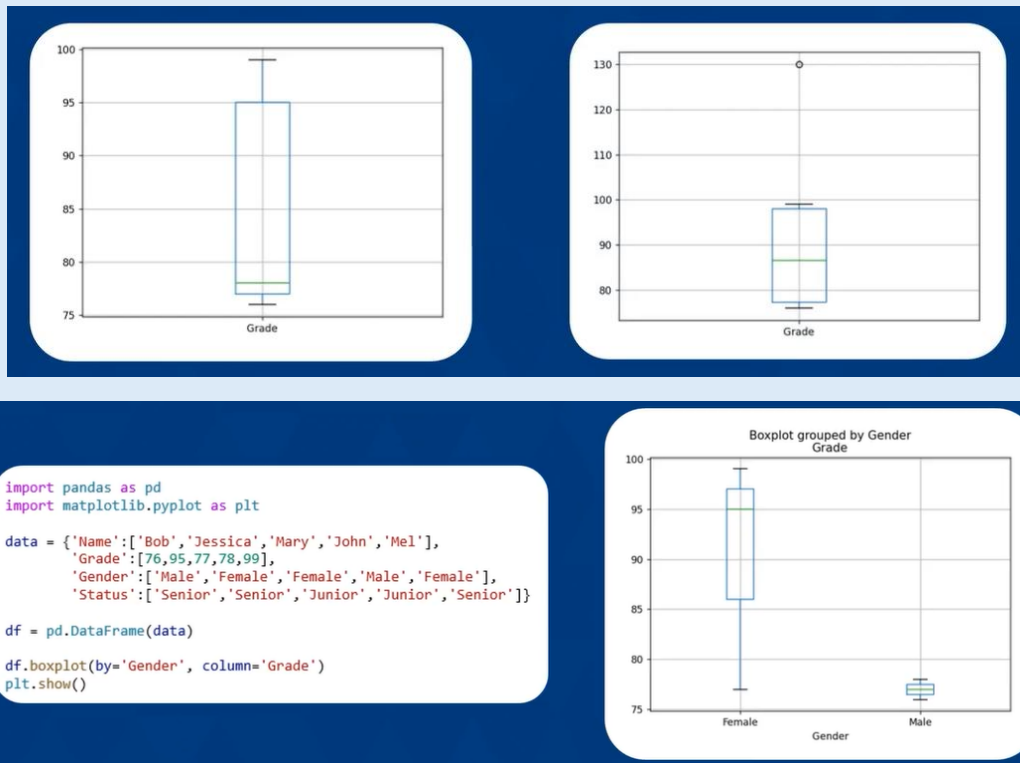
Terminal Output:

Name	Grade
Bob	76
Jessica	83
Mary	77
John	78
Mel	95

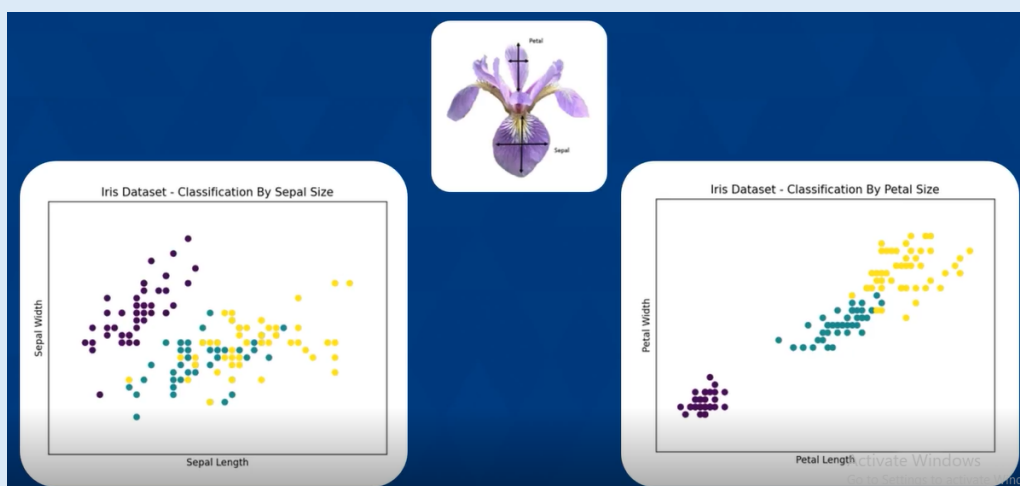


Data Visualisation and Matplotlib

- Exploratory data analysis helps identify relationships between variables and guides the selection of features for predictive modelling.
- Boxplots and histograms effectively highlight data distributions, outliers, and differences across categories such as gender or class.



- Scatter plots reveal bivariate relationships and are especially useful for assessing variable relevance in classification tasks like the Iris dataset.



- Pivot tables and group-by operations enable aggregation and comparison across multiple dimensions, aiding deeper understanding of patterns.

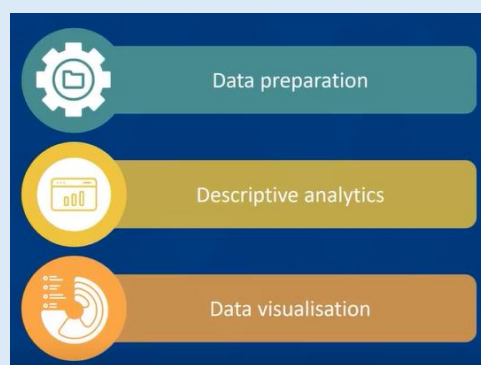
body_style											
body_style											
convertible hardtop hatchback sedan wagon											
make											
alfa-romero	2	0	1	0	0						
audi	0	0	1	5	1						
bmw	0	0	0	8	0						
chevrolet	0	0	2	1	0						
dodge	0	0	5	3	1						
honda	0	0	7	5	1						
isuzu	0	0	1	3	0						
jaguar	0	0	0	3	0						
mazda	0	0	10	7	0						
mercedes-benz	1	2	0	4	1						
mercury	0	0	1	0	0						
mitsubishi	0	0	9	4	0						
nissan	0	1	5	9	3						
peugot	0	0	0	7	4						
plymouth	0	0	4	2	1						
porsche	1	2	2	0	0						
renault	0	0	1	0	1						
saab	0	0	3	3	0						
subaru	0	0	3	5	4						
toyota	1	3	14	10	4						
volkswagen	1	0	1	9	1						
volvo	0	0	0	8	3						

symboling	normalized_losses	make	fuel_type	aspiration	num_doors	...	compression_ratio	horsepower	peak_rpm	city_mpg	highway_mpg	price
0	3	NaN	alfa-romero	gas	std	two	...	9.0	111.0	5000.0	21	27 13495.0
1	3	NaN	alfa-romero	gas	std	two	...	9.0	111.0	5000.0	21	27 16500.0
2	1	NaN	alfa-romero	gas	std	two	...	9.0	154.0	5000.0	19	26 16500.0
3	2	164.0	audi	gas	std	four	...	10.0	102.0	5500.0	24	30 13950.0
4	2	164.0	audi	gas	std	four	...	8.0	115.0	5500.0	18	22 17450.0
...	...	...	...	...	...	...	...	...	...	...	...	...
200	-1	95.0	volvo	gas	std	four	...	9.5	114.0	5400.0	23	28 16845.0
201	-1	95.0	volvo	gas	turbo	four	...	8.7	160.0	5300.0	19	25 19045.0
202	-1	95.0	volvo	gas	std	four	...	8.8	134.0	5500.0	18	23 21485.0
203	-1	95.0	volvo	diesel	turbo	four	...	23.0	106.0	4800.0	26	27 22470.0
204	-1	95.0	volvo	gas	turbo	four	...	9.5	114.0	5400.0	19	25 22525.0

- Visual insights from pie charts, histograms, and grouped scatter plots simplify interpretation and support informed machine learning decisions.

Module Summary

- Python's built-in data structures lack efficiency for advanced analytics, prompting the use of specialised libraries for better performance.
- Libraries like NumPy, Pandas, and Matplotlib offer robust tools for data preparation, visualisation, and machine learning modelling.
- Seamless integration among libraries creates a unified development environment for data scientists and AI engineers.
- Capabilities include handling complex datasets, generating descriptive statistics, and building predictive models with minimal overhead.



- Continued exploration of these libraries enhances proficiency in AI solutioning and supports scalable, real-world data applications.

THE END